# Module 10: Develop Ansible Playbooks in AAP

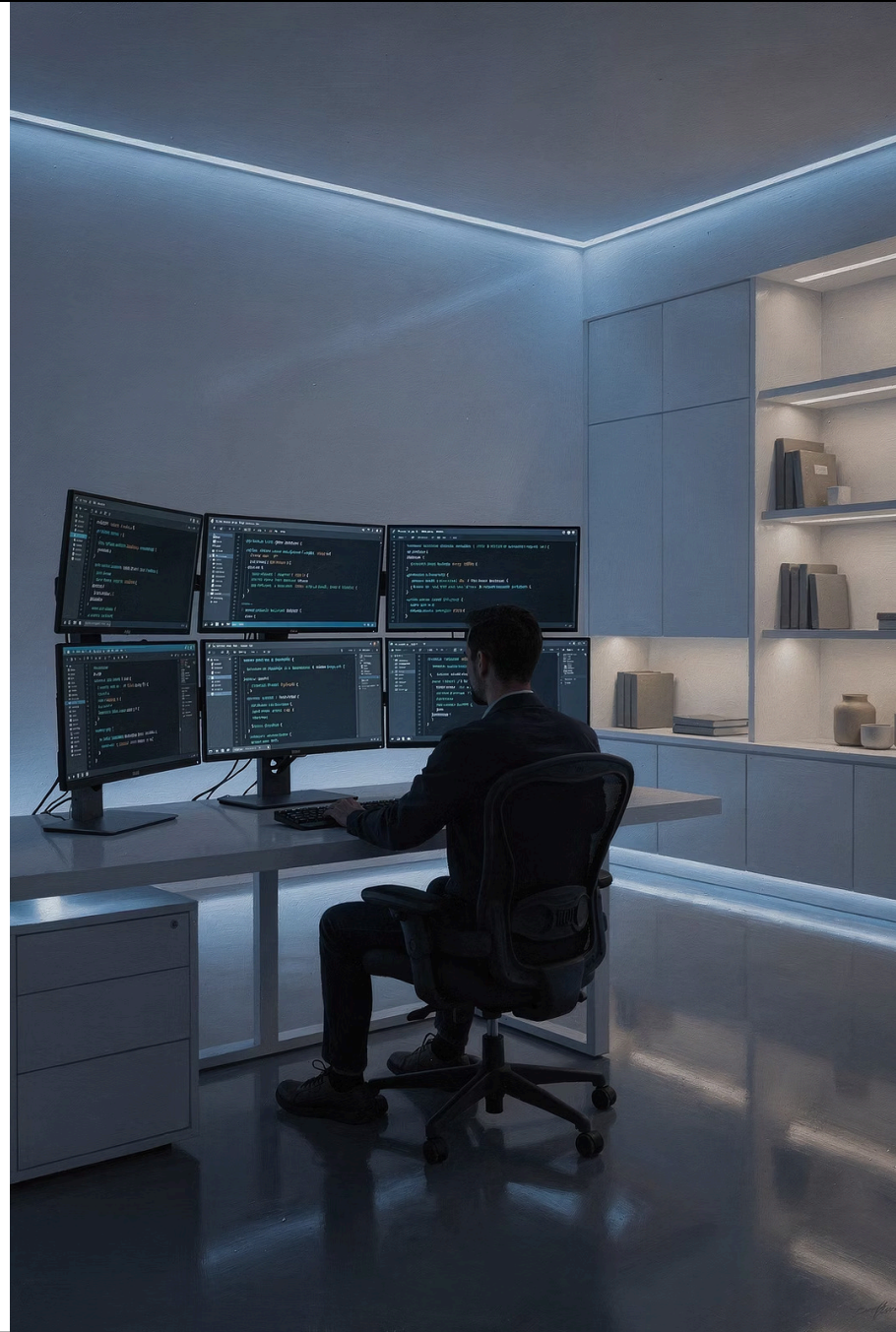
**Duration: 4 Hours**

Theory: 30%

Hands-on: 70%

## Learning Outcomes

- Write **Ansible playbooks from scratch**
- Use **variables and facts**
- Follow **enterprise playbook best practices**
- Use **advanced constructs**
- Integrate playbooks with **AAP Controller features**



# 10.1 Ansible Playbook Fundamentals

## What is a Playbook?

A **playbook** is the **automation script used by Ansible**. It defines what tasks to run, on which hosts, in what order, and using which modules. Playbooks are written in **YAML**.

### Bash

Shell Script

### PowerShell

PS Script

### Ansible

Playbook

### Example Playbook

```
---
- name: Install and start NGINX
 hosts: webservers
 become: true
 tasks:
 - name: Install nginx
 ansible.builtin.package:
 name: nginx
 state: present
 - name: Start nginx service
 ansible.builtin.service:
 name: nginx
 state: started
```

# Real Enterprise Example

Instead of running **20 manual commands**, a single playbook runs everything automatically — from provisioning to deployment.

1. **Install Web Server**
2. **Configure Firewall**
3. **Deploy Application**
4. **Start Service**

A playbook may automate server provisioning, patching, deploying applications, configuring firewalls, and creating users — all in one automated run.

# YAML Syntax Basics for Beginners

Ansible uses **YAML (YAML Ain't Markup Language)**. Understanding its rules is essential for writing correct playbooks.

## Rule 1: Indentation Matters

### ✅ Correct

```
tasks:
  - name: Install nginx
```

### ❌ Incorrect

```
tasks:
- name: Install nginx
```

Always use **2 spaces** for indentation — never tabs. YAML is whitespace-sensitive and incorrect indentation will cause parse errors.

# YAML Rule 2: Key-Value Pairs

Key-value pairs are the most fundamental building block in YAML. Each key is separated from its value by a colon and a space.

```
name: Install nginx
state: present
```

### Key

The identifier on the left side of the colon — e.g., `name`, `state`

### Value

The data on the right side — e.g., `Install nginx`, `present`

### Separator

Always a colon followed by a space: `key: value`

# YAML Rule 3: Lists

Lists in YAML are defined using a dash (`-`) followed by a space before each item. Lists are used extensively in Ansible for defining packages, tasks, and hosts.

```
packages:
  - nginx
  - git
  - curl
```

### nginx

Web server

### git

Version control

### curl

HTTP client

# YAML Rule 4: Dictionaries

Dictionaries (also called mappings) group related key-value pairs under a single parent key. They are used in Ansible to define structured data like user configurations or module parameters.

```
user:
  name: devops
  shell: /bin/bash
```

### Parent Key

`user` — the dictionary name that groups related fields

### Child Keys

`name` and `shell` — indented fields belonging to the parent

# YAML Tips at a Glance

| Rule | Example |
|---|---|
| Use spaces, not tabs | Always indent with spaces |
| Indentation | 2 spaces per level |
| Lists start with | `-` (dash + space) |
| Key-value separator | `key: value` |

# Playbook Structure: Plays and Tasks

A playbook contains **plays**. Each play targets **hosts**. Inside a play are **tasks**.

> ### Playbook
>
> The top-level YAML file containing one or more plays

> ### Play
>
> Targets a group of hosts and defines variables and tasks

> ### Tasks
>
> Individual units of work executed in sequence

```yaml
---
- name: Configure web servers
  hosts: webservers
  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present
```

| Section | Meaning |
| --- | --- |
| `name` | Description of the play |
| `hosts` | Target machines |
| `tasks` | Work to perform |

# Understanding Modules

Modules are **the building blocks of Ansible** — they perform the actual work in every task.

### file

Manage files and directories

### copy

Copy files to remote hosts

### service

Manage system services

### package

Install software packages

### user

Create and manage system users

```
- name: Create directory
  ansible.builtin.file:
  path: /opt/app
  state: directory
```

COMMON MODULES

# Copy Module

The **copy** module is used to copy files from the control node to managed hosts. It is one of the most frequently used modules for distributing configuration files.

```
- name: Copy config file
  ansible.builtin.copy:
    src: config.conf
    dest: /etc/app/config.conf
```

### src

Source file path on the control node

### dest

Destination path on the managed host

COMMON MODULES

# File Module

The **file** module manages file and directory properties on remote hosts — including creation, deletion, permissions, and ownership.

```
- name: Create directory
  ansible.builtin.file:
    path: /opt/app
    state: directory
```

### path

The file or directory path to manage

### state

Desired state: `directory`, `file`, `absent`, `touch`

COMMON MODULES

# Service Module

The **service** module controls system services — starting, stopping, restarting, or enabling them on boot across managed hosts.

```
- name: Start nginx
  ansible.builtin.service:
    name: nginx
    state: started
```

### started

Ensure the service is running

### stopped

Ensure the service is not running

### restarted

Restart the service

### enabled

Start automatically on boot

COMMON MODULES

# Package Module

The **package** module installs, updates, or removes software packages. It is distribution-agnostic, working across `yum`, `apt`, and other package managers automatically.

```
- name: Install nginx
  ansible.builtin.package:
    name: nginx
    state: present
```

### present

Ensure the package is installed

### absent

Ensure the package is removed

### latest

Ensure the latest version is installed

COMMON MODULES

# User Module

The **user** module creates and manages system user accounts on remote hosts — including shell, home directory, groups, and password settings.

```
- name: Create user
  ansible.builtin.user:
    name: devops
    shell: /bin/bash
```

### name

The username to create or manage

### shell

The login shell for the user

### state

`present` to create, `absent` to remove

# Writing Your First Playbook

```yaml
---
- name: Install nginx web server
  hosts: all
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present
    - name: Ensure nginx running
      ansible.builtin.service:
        name: nginx
        state: started
```

### Steps to Run

1. Create the file: `nano install_nginx.yml`
2. Paste the playbook content
3. Run: `ansible-playbook install_nginx.yml`

In **AAP**, playbooks are executed through **Job Templates** — not the command line directly.

# 10.2 Variables and Facts

## What are Variables?

Variables store values used in playbooks, avoiding hardcoded data. They make playbooks reusable and easier to maintain across environments.

```
app_port: 8080
```

Instead of hardcoding values directly in tasks, reference a variable that can be changed in one place.

### Why Use Variables?

- Reuse playbooks across environments
- Change values without editing tasks
- Improve readability and maintainability

# Variables: Bad vs. Better Practice

Hardcoding values directly in tasks makes playbooks brittle and difficult to reuse. Using variables is always the better approach.

### ❌ Bad Practice

```
port: 8080
```

Value is hardcoded — changing it requires editing every task that references it.

### ✅ Better Practice

```
port: "{{ app_port }}"
```

Value is referenced via a variable — change it once in the vars section and it updates everywhere.

# Defining Variables

Variables can be defined in several places. The most common for beginners is directly inside the playbook using the `vars` key.

```
---
- hosts: all
  vars:
    app_port: 8080
  tasks:
    - name: Show port
      debug:
        msg: "Port is {{ app_port }}"
```

### vars block

Defined at the play level, available to all tasks in that play

### Jinja2 syntax

Reference variables using `{{ variable_name }}` double curly braces

### debug module

Prints messages during playbook execution for troubleshooting

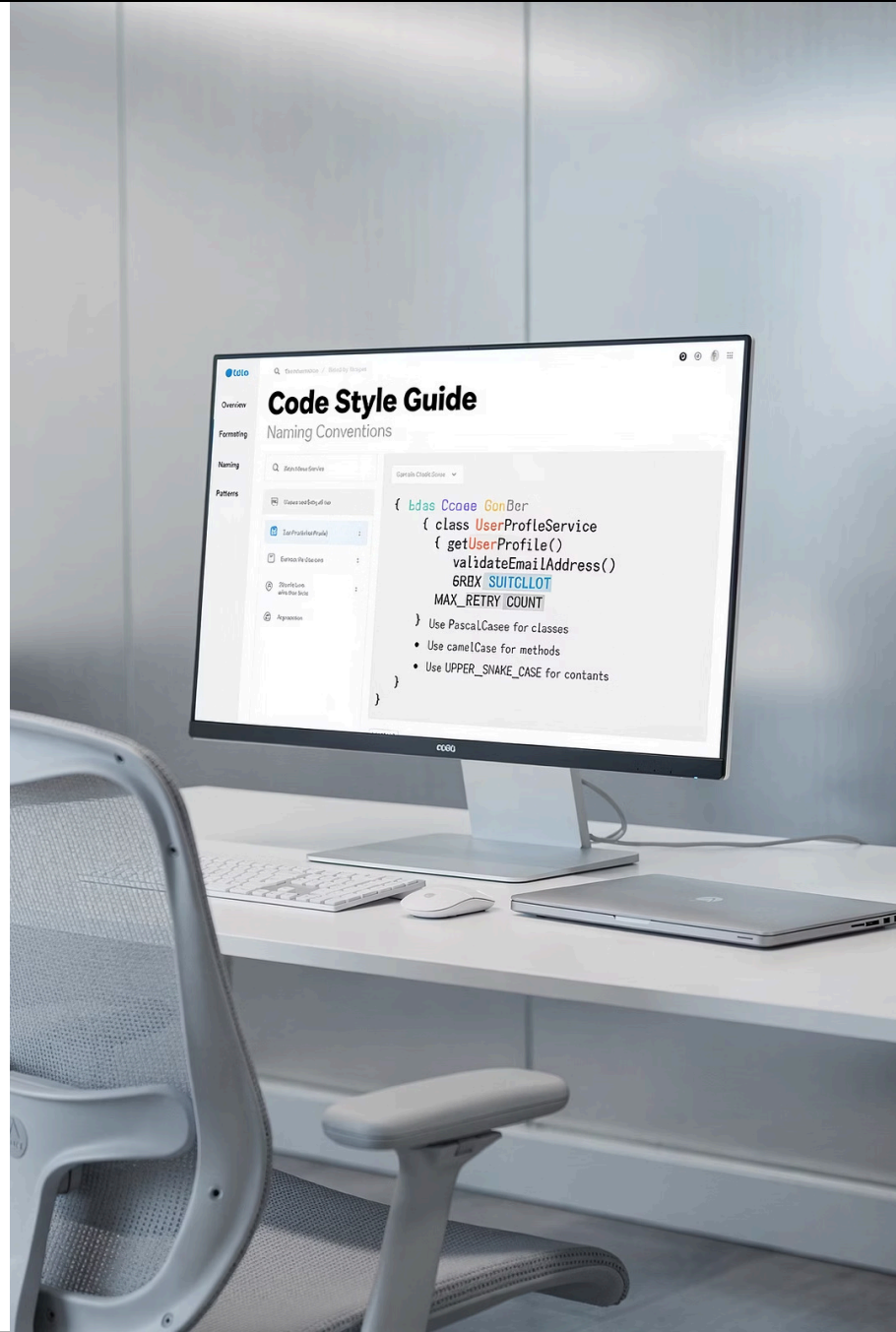
# Variable Naming Conventions

## ✅ Use These

- Lowercase letters
- Underscores to separate words

```
app_port
db_host
web_server_count
```

## ❌ Avoid These

- CamelCase: `AppPort`
- Hyphens: `app-port`
- All caps: `APPPORT`

Consistent naming prevents errors and makes playbooks easier to read and maintain across teams.